WEEK 12 — SOC FINAL CAPSTONE: THREAT DETECTION & INCIDENT RESPONSE

A multi-stage incident (reconnaissance, authentication events, system and file activity) detected, correlated and handled with Wazuh and Snort in an authorised lab

Candidate	Md Mohtasim (DG/JULY/CYBER/017)
Module	Security Operations Center (SOC) — Final Capstone: Threat Detection & Incident Response
Scenario	Multi-stage incident (Reconnaissance, Authentication, System/File activity)
Monitored endpoint	Ubuntu endpoint with Wazuh agent (agent name: ubuntu-agent)
Environment	Local lab — VirtualBox, host-only network 192.168.56.0/24
Primary tools	Wazuh , Snort, OpenSSH, Wazuh File Integrity Monitoring

1. Introduction

This is the final capstone of the four-week SOC module. Week 9 produced a network sensor (Snort), Week 10 added a SIEM (Wazuh) that collects host evidence, and Week 11 practised correlating the two. Week 12 asks for the whole analyst workflow on **one incident**: detect it, investigate it, classify it, correlate network and host evidence, extract indicators of compromise (IOCs), respond, write a ticket, measure the result and reflect. The same private lab is reused. Every event was generated deliberately by the author on virtual machines the author owns, inside an isolated host-only network, and nothing was directed at any third-party or production system. No real personal data is included.

The scenario is **Scenario D — Multi-stage incident**, the recommended advanced option. It combines three authorised lab activities that were validated in earlier weeks: reconnaissance (ICMP probing seen by Snort), authentication events (failed SSH logins followed by a success) and system/file activity (a session, sudo to root and a change in a monitored folder seen by Wazuh).

1.1 Objectives

- Verify that the SOC lab (Ubuntu endpoint, Wazuh manager, agent, dashboard, Snort) is working.
- Simulate one safe, controlled multi-stage incident inside the author's own lab.
- Document the detection with exact fields (time, IPs, account, protocol, port, SID, rule ID, severity, source).
- Investigate like an L1/L2 analyst and classify the result (true positive, false positive, benign true positive or inconclusive).
- Correlate Snort and Wazuh into one timeline and explain what each tool adds.
- Build an IOC register, a six-phase response plan, the ticket DG-SOC-2026-001 and a metrics summary.
- Reflect on the module and propose improvements.

1.2 How this report handles evidence

Every ID, address and count is copied from raw alerts and nothing is invented. This report therefore uses only values that appear in the author's own Week 11 lab evidence (rule IDs, levels, agent name, interface, versions, dashboard counts). Values that were masked or not recorded are left as placeholders [like this] to be filled from the raw alert JSON before submission. The illustrative mock-up images printed in the Week 12 guide are not used as evidence anywhere in this report.

1.3 Terminology

Term	Meaning in this report
Event	Any recorded occurrence, for example a line in /var/log/auth.log.
Alert	An event or group of events that matched a Wazuh rule or a Snort signature.
Incident	Confirmed alerts that violate policy and need a response. Here: DG-SOC-2026-001.
Benign true positive	The rule correctly detected real activity, but the activity was authorised (the author's own test).
IOC	An observable artefact (IP, account, port, path, rule, timestamp) used to find or block related activity.

2. SOC Architecture

A SOC combines sensors that collect telemetry, a platform that normalises and correlates it, analysts who triage and investigate, and a process that turns confirmed findings into response actions. In this lab the roles are filled as follows.

SOC layer	Lab component	Function

Network sensor (IDS)	Snort, passive afpacket DAQ on enp0s8	Inspects packets against rules; reports what crossed the wire.
Endpoint sensor	Wazuh agent on the Ubuntu endpoint (ubuntu-agent)	Ships auth.log, sudo, PAM and File Integrity Monitoring (syscheck) events.
SIEM	Wazuh manager + indexer + dashboard (4.14.8)	Decodes events, applies rules, stores alerts and lets the analyst search and pivot.
Analyst workspace	Wazuh dashboard, Snort console/alert log	Alert review, raw JSON inspection, filtering by source IP and time.
Process	Playbooks, ticket DG-SOC-2026-001	Classification, escalation, containment and documentation.

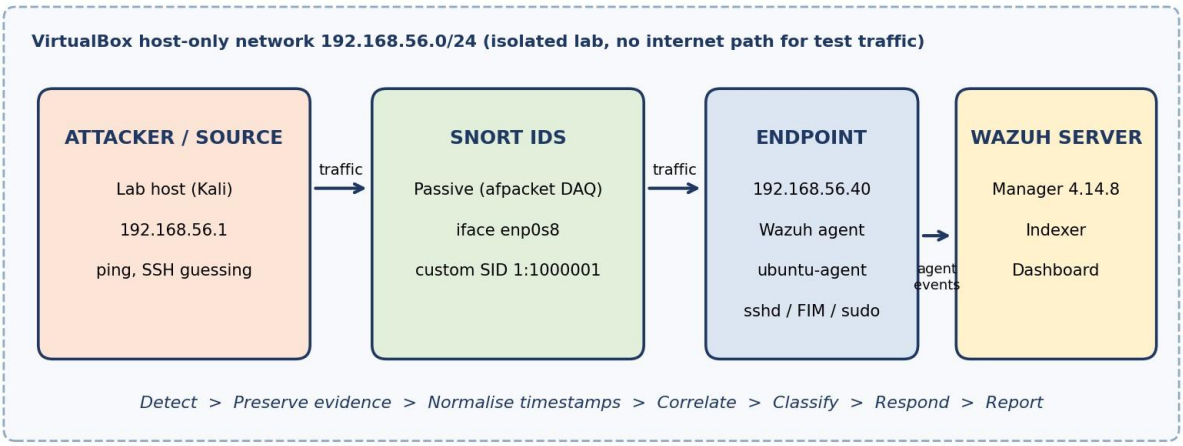


Figure 1 — Lab architecture used for the capstone (Snort is a passive sensor on interface enp0s8)

2.1 Detection versus response workflow

Threat detection is the continuous collection of telemetry, execution of rules and triage of alerts. **Incident response** begins when an incident is confirmed. The capstone follows one continuous chain: detect, preserve evidence, normalise timestamps, correlate, classify, respond, report. Evidence is preserved before any interpretation or clean-up so that the report stays defensible.

2.2 Why two sensors

Snort inspects packets and sees reconnaissance and other network behaviour, but it cannot read encrypted SSH payloads and knows nothing about what happened on the host. Wazuh reads logs and file state on the endpoint and therefore knows whether authentication succeeded and what changed, but it only sees the network if the host logs it. Each tool has blind spots that the other covers, which is the central argument of Section 10.

3. Lab Environment and Component Validation (Task 1)

The environment built in Weeks 9 and 10 was reused. The table records the roles and the values documented in the lab evidence; items not yet recorded are marked for completion.

Component	Role	Value / status in this lab
Hypervisor / network	VirtualBox host-only network	192.168.56.0/24, isolated from production and the internet for test traffic

Ubuntu endpoint	Test endpoint, monitored victim	192.168.56.40, Wazuh agent name ubuntu-agent, SSH enabled
Source host	Generates test traffic	Lab Kali host, 192.168.56.1 (see note below)
Wazuh manager	Receives agent data, applies rules	Version 4.14.8, status Active, services wazuh-manager and wazuh-indexer active (Figure 3)
Wazuh dashboard	Search and investigation UI	Accessible; Security events and Discover views used (Figures 2, 4, 5)
Snort IDS	Network IDS	Interface enp0s8, afpacket DAQ passive, 300 appid patterns loaded, custom rule SID 1:1000001 (Figure 6)
Time sync	Needed for correlation	[confirm NTP/timezone on all VMs: TZ = ____]

Note on 192.168.56.1. In a default VirtualBox host-only network, .1 is normally the host-side adapter, not a guest VM. Before submission, confirm in the raw alert (data.srcip) and with `ip addr` on the Kali VM which address generated the traffic, and correct the report if it differs.

3.1 Validation evidence

The Wazuh overview and manager pages were captured from the working lab. Figure 2 shows the alert counts by severity for the last 24 hours; Figure 3 shows the manager status.

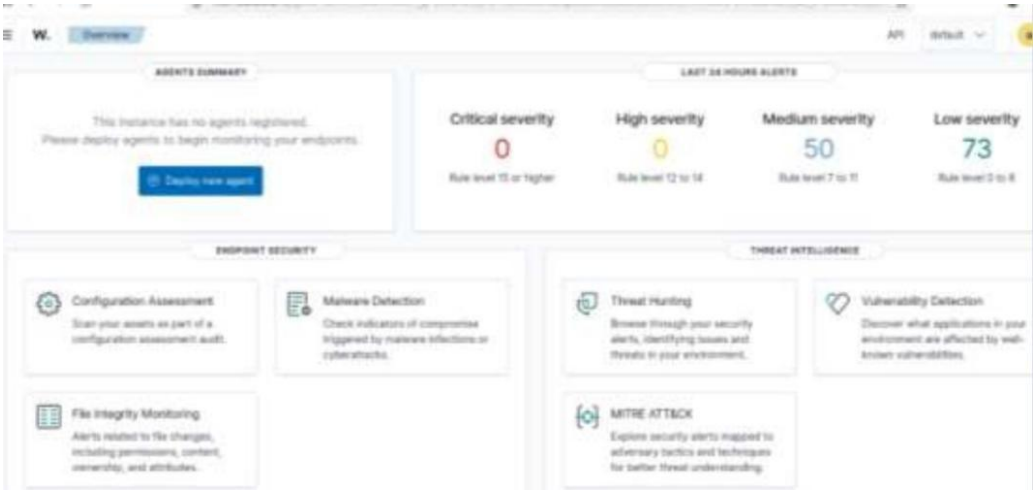


Figure 2 — Wazuh dashboard overview, last 24 h: 0 critical, 0 high, 50 medium, 73 low



Figure 3 — Wazuh manager status: Active, version 4.14.8, uptime 2d 4h 32m, wazuh-manager and wazuh-indexer services active

Evidence still to capture (E02). The Agents Summary tile in Figure 2 reads “no agents registered”, while alerts in Figures 4 and 5 clearly come from ubuntu-agent. A fresh screenshot of the Wazuh Agents page must be inserted here, showing ubuntu-agent with its ID, status Active, IP 192.168.56.40 and last keep-alive.

3.2 Validation commands

Standard checks for the components above. Run them on the matching VM and attach the output as evidence; do not paste passwords or tokens.

Check	Command (on the VM where the component runs)	Expected result
Wazuh manager	sudo systemctl status wazuh-manager	active (running)
Wazuh indexer	sudo systemctl status wazuh-indexer	active (running)
Wazuh dashboard	sudo systemctl status wazuh-dashboard	active (running)
Agent on endpoint	sudo systemctl status wazuh-agent	active (running), connected to manager
Snort version	snort -V	Version banner
Interface	ip addr	enp0s8 up with 192.168.56.x address

4. Wazuh Setup

Wazuh was deployed in Week 10 as an all-in-one server (manager, indexer and dashboard) with a single Ubuntu agent. This section summarises the configuration that matters for the capstone.

4.1 Components and data flow

- **Agent** on the Ubuntu endpoint reads /var/log/auth.log (sshd, PAM and sudo events) and runs syscheck (File Integrity Monitoring) on configured directories.
- **Manager** decodes each event, matches it against the rule set and assigns a rule ID and level.
- **Indexer** stores alerts so they can be searched; the dashboard provides Security events, Discover and module views (FIM, MITRE ATT&CK, Threat Hunting).
- **Active response** was deliberately left disabled in the lab, so containment in this incident is a documented analyst action.

4.2 Rules relevant to this incident

Rule ID	Level	Description	Source	Dashboard band
5710	5	sshd: attempt to login using a non-existent user	auth.log	Low (0–6)
5760	5	sshd: authentication failed (valid user, wrong password)	auth.log	Low (0–6)
5711	7	Failed authentication for user (observed, Figure 4)	auth.log	Medium (7–11)
5763	10	sshd: brute force trying to get access to the system	auth.log (frequency rule)	Medium (7–11)
5715	3	sshd: authentication success	auth.log	Low (0–6)
5501	3	PAM: login session opened	auth.log (PAM)	Low (0–6)
5402	3	Successful sudo to ROOT executed	auth.log (sudo)	Low (0–6)
554	5	File added to the system (monitored folder)	syscheck (FIM)	Low (0–6)
550	7	Integrity checksum changed	syscheck (FIM)	Medium (7–11)

Rule IDs and levels are the Wazuh defaults used in Week 11; the dashboard values take precedence if they differ. Band boundaries follow the dashboard tiles in Figure 2 (critical 15+, high 12–14, medium 7–11, low 0–6).

4.3 Two severity scales

The tool severity (rule level) and the analyst severity are not the same thing. For example, rule 5715 (a successful login) is level 3 and sits in the Low band, but a success that follows a brute-force burst from the same source is a **High**-severity analyst finding. This report therefore records both the Wazuh level and the analyst rubric (Section 14) for every alert.

4.4 Security events screenshot

The Week 11 security-events screenshots show the rule fields an analyst reads first (agent, rule ID, level, description, user, source IP, location). The source IP is masked in the screenshot, so the raw JSON must be used for the exact value.

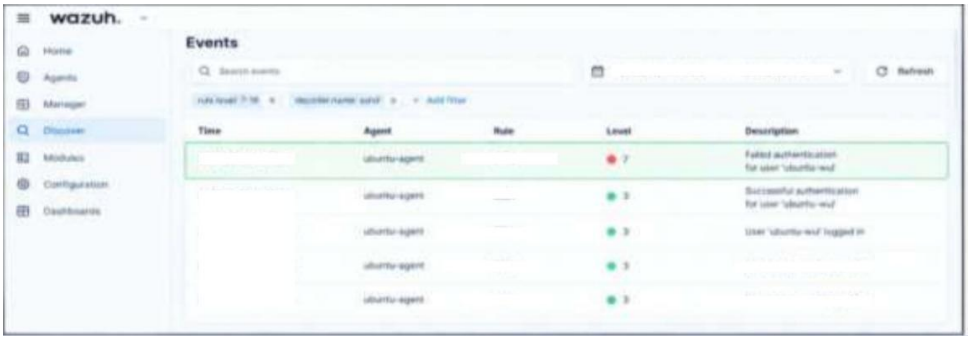


Figure 4 — Wazuh Security events: rule 5711, level 7, agent ubuntu-agent, location /var/log/auth.log

5. Snort Setup

Snort from Week 9 runs as a passive network IDS. The console output confirms that the afpacket DAQ is configured to passive, 300 application-identification patterns are loaded and packet processing is attached to interface enp0s8. The custom rule with SID 1:1000001 raises “CUSTOM ICMP Ping Detected” for every ICMP echo it sees between lab hosts.

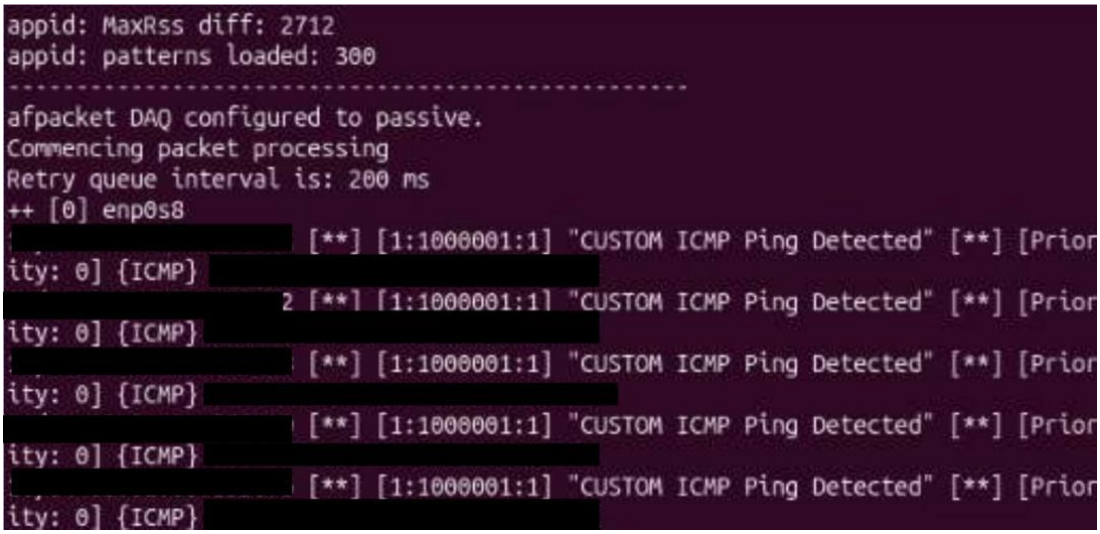


Figure 5 — Snort console: repeated “CUSTOM ICMP Ping Detected” alerts, rule 1:1000001:1, priority 0, protocol ICMP

5.1 Reading a Snort alert line

Field	Example in this lab	Meaning
Rule identifier	[1:1000001:1]	generator : SID : revision. SIDs of 1,000,000 and above are local (custom) rules.
Message	CUSTOM ICMP Ping Detected	Signature text written in the rule.

Field	Example in this lab	Meaning
Priority	0	As recorded in the console output of the lab rule.
Protocol	ICMP	Transport protocol of the packet that matched.
Source / destination	[src IP → dst IP] (masked in screenshot)	Take the exact addresses from the alert log or console, not from memory.

5.2 Coverage and limits

- The Week 9 rule set covers ICMP and HTTP only. There is **no Snort rule for SSH**, so SSH password guessing produces no Snort alert. This is a detection gap, not a sensor failure.
- SSH payloads are encrypted, so even a perfect rule could count connection attempts but could not read the password result.
- "Snort is running" does not prove it saw the test traffic; the alert in Figure 5 is the proof for the ICMP stage only.
- The Snort host and alert-log path should be recorded as part of the evidence: [confirm Snort host and alert log path].

5.3 Verification

To prove that the sensor sees the intended traffic, generate a small known stimulus (for example three pings from the source host to the endpoint) and confirm three new alerts with SID 1:1000001 appear with the expected source and destination. Record the time of the stimulus and the time of the alerts for the timeline.

6. Incident Scenario

Scenario D — Multi-stage incident. Three previously validated, low-risk lab activities are combined in a fixed order with clear timestamps. All traffic stays inside 192.168.56.0/24 and targets only the author's own endpoint.

Stage	Activity (authorised, own lab)	Expected telemetry	Primary tool
1. Reconnaissance	Limited ICMP probing from the source host to the Ubuntu endpoint	Snort alert SID 1:1000001	Snort
2. Authentication	A small, intentional number of failed SSH logins, then one successful login with the lab account	Wazuh 5710 / 5760 / 5711, frequency alert 5763, success 5715	Wazuh
3. System / file activity	After login: session, sudo to root, and creation then modification of one harmless file in the monitored lab folder	Wazuh 5501, 5402, FIM 554 and 550	Wazuh (syscheck)

6.1 Rules of engagement followed

- Only VMs and IP addresses owned by the author, on an isolated host-only network; no public IPs or production systems.
- A dedicated lab account and test directory; no real credentials or personal data.
- Narrow and short: a small number of failed logins, stop immediately after confirming the logs; no malware, persistence or credential theft.
- Actual date, time, timezone, hostname, IP, command and outcome recorded for each stage.
- Raw event details preserved before interpretation; no SID, rule ID or IP invented.

6.2 Actions performed

The file-activity stage uses harmless commands adapted to the monitored folder configured in the lab.

Replace the path with the directory actually monitored by your FIM configuration.

Step	Command / action	Purpose
a	`ping -c 3 192.168.56.40` (from the source host)	Stage 1: produce Snort ICMP alerts
b	A few intentional wrong-password SSH attempts to the lab account, then stop	Stage 2: produce 5710/5760/5711 and the 5763 burst
c	One correct SSH login with the lab account	Stage 2: produce 5715 and 5501
d	`sudo -l` or a harmless sudo command	Stage 3: produce 5402
e	`mkdir -p /tmp/soc-lab-demo;echo 'SOC capstone baseline' > /tmp/soc-lab-demo/evidence.txt;echo 'controlled modification' >> /tmp/soc-lab-demo/evidence.txt`	Stage 3: produce FIM 554 then 550 (path must be inside the monitored directories)
f	`stat /tmp/soc-lab-demo/evidence.txt`	Record time, size and owner for the evidence table

The report lists /tmp/soc-lab-demo/evidence.txt as the example file path. If your FIM monitors a different folder, replace it everywhere (Sections 9, 11, 13). **Evidence still to capture (E04):** a terminal screenshot of the commands above with date/time and hostname visible.

7. Detection (Task 3)

The detection record below copies the fields that are known from the lab evidence. Fields that were masked or not captured are shown as placeholders to be filled from the raw alert (Wazuh Raw JSON tab; Snort alert log). If a field is truly absent from the event, write “N/A — not present in source event”.

Field	Stage 1: Snort	Stage 2: Wazuh authentication	Stage 3: Wazuh system / file
Detection time + TZ	[hh:mm:ss TZ]	[hh:mm:ss TZ] first failure; [ ] 5763; [ ] 5715	[hh:mm:ss TZ]
Source IP	192.168.56.1 [confirm]	192.168.56.1 [confirm in data.srcip]	N/A (local host activity)
Destination IP	192.168.56.40	192.168.56.40	N/A
User / account	N/A	root; also standard account ubuntu-wui in failed-auth events	[account that ran sudo and created the file]
Protocol	ICMP	TCP (SSH)	N/A (host log)
Port	N/A (ICMP)	22	N/A
Snort SID	1:1000001 “CUSTOM ICMP Ping Detected”	None (no SSH rule)	None
Wazuh rule ID	None	5710, 5760, 5711, 5763, 5715	5501, 5402, 554, 550
Rule level (tool)	Priority 0	5, 5, 7, 10, 3	3, 3, 5, 7
Analyst severity	Low	Medium; High for 5763 and 5715	Low (5501), Medium (5402, 554, 550)

Detection source	Snort	Wazuh (/var/log/auth.log)	Wazuh (auth.log, syscheck FIM)
Affected asset	Ubuntu endpoint	ubuntu-agent, 192.168.56.40	ubuntu-agent, 192.168.56.40

7.1 What made the activity suspicious?

- **Sequence.** Probing, then repeated authentication failures, then a success for the same account from the same address, then privileged use. Each step alone is low value; together they match a successful password-guessing attack.
- **Frequency.** Several failures within a short window crossed the brute-force threshold and raised frequency rule 5763 (level 10).
- **Privileged target.** The account of concern is root and the follow-up included sudo to root.
- **Protocol exposure.** SSH on port 22 with password authentication is a common attack target.
- **Post-login change.** A file was added and then changed in a monitored folder after login, the point at which an attacker would stage tools or persistence.

7.2 Raw alert evidence

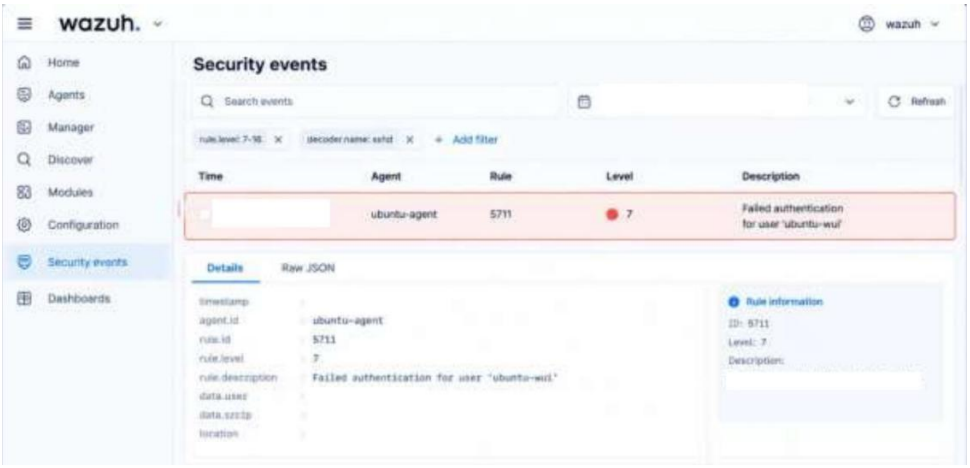


Figure 6 — Authentication alert timeline in Discover: one failed login (level 7) followed by successful authentication and user-logged-in events

Evidence still to capture (E05b): Wazuh Raw JSON for rule 5763 (or 5715) showing data.srcip, data.dstuser, rule.id, rule.level, agent.name and timestamp.

8. Alert Analysis and Full Investigation (Task 4)

The investigation follows the L1 triage routine: acknowledge, validate the source tool and raw fields, identify the asset, check related events before and after, classify, assess severity, decide on escalation and document everything.

Investigation question	Finding
What happened?	A source host in the lab probed the endpoint, made repeated failed SSH logins that crossed the brute-force threshold, then authenticated successfully. A session opened, sudo to root ran, and a file in the monitored lab folder was created and modified.
When did it happen?	Start: first Snort ICMP alert or first failed SSH login, whichever is earlier [hh:mm:ss TZ]. End: last FIM event [hh:mm:ss TZ]. All times normalised to one timezone (source TZ: [ ]).
Which system was affected?	Ubuntu endpoint, agent ubuntu-agent [agent ID], 192.168.56.40.

Which IP generated the activity?	Lab source host 192.168.56.1 [confirm in raw event].
Which account was involved?	root (account of concern) and the standard account ubuntu-wui seen in failed-authentication events.
Which alerts were generated?	Snort: 1:1000001. Wazuh: 5710, 5760, 5711, 5763, 5715, 5501, 5402, 554, 550.
Which Snort rule triggered?	SID 1:1000001 “CUSTOM ICMP Ping Detected” (ICMP stage only). No Snort rule covers SSH.

Investigation question	Finding
Which Wazuh rules triggered?	5710/5760/5711 for individual failures, 5763 for the burst, 5715 for the success, 5501 and 5402 for the session and sudo, 554 and 550 for the file.
What evidence was collected?	Wazuh alert JSON, raw /var/log/auth.log lines, dashboard timeline filtered on the source IP, Snort console/alert log, FIM event details, terminal record of commands (see Appendix B).
True positive or false positive?	**Benign true positive.** Every rule detected the behaviour it was written for, and the activity matches the planned lab test (source, timing and account). Not malicious, because it was authorised.
Potential impact (production view)	A successful root-level compromise: loss of confidentiality and integrity of the host, persistence, lateral movement. In the lab, impact is none beyond the harmless test file; no malware or unexpected file change outside the test folder was seen.

8.1 Classification reasoning

Detection is not the same as maliciousness. A rule can correctly detect a brute-force pattern that the analyst performed on purpose. The decision uses three checks: (1) does the raw event match what the rule claims, (2) is the source on the approved lab asset list and inside the test window, and (3) is there any activity outside the plan. All three support **benign true positive**. In production the same pattern would be escalated to L2 as a likely compromise because success followed the failures.

8.2 Alert-by-alert review

Alert	Tool	Tool level	Analyst severity	Classificatio n	Reason
ICMP ping (1:1000001)	Snort	Priority 0	Low	Benign TP	Common diagnostics; reconnaissance only in sequence
5710 non-existent user	Wazuh	5	Medium	Benign TP	Guessing of an invalid account name
5760 wrong password	Wazuh	5	Medium	Benign TP	Valid user, repeated failures
5711 failed authentication	Wazuh	7	Medium	Benign TP	Failed login for user ubuntu-wui (Figure 4)
5763 brute force	Wazuh	10	High	Benign TP	Threshold crossed from one source
5715 authentication success	Wazuh	3	High	Benign TP	Success after a failure burst; treat account as compromised until verified
5501 PAM session opened	Wazuh	3	Low	Benign TP	Normal consequence of login
5402 sudo to ROOT	Wazuh	3	Medium	Benign TP	Privilege use after suspicious login
554 file added	Wazuh FIM	5	Medium	Benign TP	New file in monitored folder; matches test

550 checksum changed	Wazuh FIM	7	Medium	Benign TP	Controlled modification; matches test

9. Incident Timeline

The timeline below orders the stages. Replace each [hh:mm:ss TZ] with the exact timestamp from the raw alert after converting all sources to one timezone. Order is taken from the lab evidence; do not assume causality from timing alone.

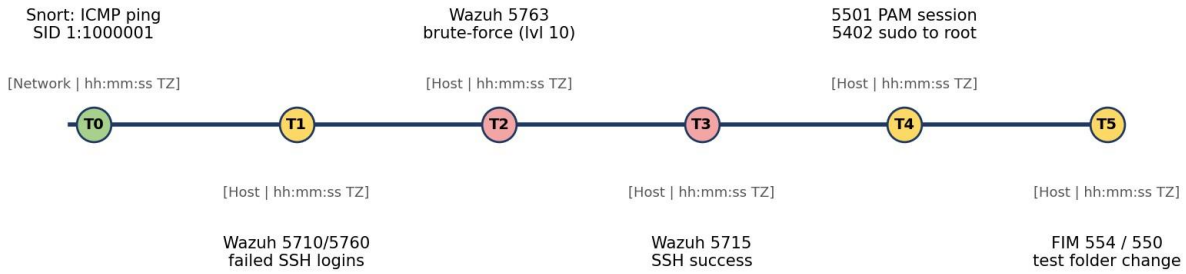


Figure 7 — Incident timeline T0–T5 (colour = analyst severity: green low, yellow medium, red high)

Time (TZ)	Event	Source	Finding / confidence
T0 [hh:mm:ss]	ICMP echo from 192.168.56.1 to 192.168.56.40	Snort	SID 1:1000001; network recon, Low. High confidence (direct packet evidence).
T1 [hh:mm:ss]	First failed SSH login (non-existent user / wrong password)	Wazuh	5710 / 5760 / 5711; authentication failures, Medium.
T2 [hh:mm:ss]	Failure threshold crossed	Wazuh	5763 level 10; brute-force pattern, High.
T3 [hh:mm:ss]	Successful SSH authentication, same account and source	Wazuh	5715; requires investigation, High.
T4 [hh:mm:ss]	PAM session opened; sudo to root	Wazuh	5501, 5402; privilege use, Medium.
T5 [hh:mm:ss]	File created then modified in monitored lab folder	Wazuh FIM	554, 550; system/file activity, Medium.
T6 [hh:mm:ss]	Analyst decision	Analyst	Benign true positive, authorised test; ticket DG-SOC-2026-001 opened.

9.1 Time normalisation

Wazuh stores timestamps in the manager's configured timezone and Snort uses the sensor host's clock. Before correlating, record the timezone of each source and check for VM clock drift. A drift of even a few seconds can reorder T0 and T1. If the clocks disagree, record the offset and state it in the ticket.

Evidence still to capture (E07): timeline screenshot — Wazuh Discover filtered by data.srcip and the test time window, with the Snort alert log lines for the same window beside it.

10. Snort + Wazuh Correlation (Task 5)

Correlation links alerts that share a source IP, destination, account, host or time window so that separate alerts tell one story. It is strongest when timestamps, host identity, addresses and event context all align.

Stage	Activity	Snort	Wazuh
Recon	ICMP ping	Alert (1:1000001)	No alert
Auth	Failed SSH logins	No rule (gap)	5710 / 5760 / 5711 / 5763
Auth	SSH success	No (encrypted)	5715
Post-login	Session + sudo	No	5501 / 5402
File	Monitored-folder change	No (not network)	FIM 554 / 550

Figure 8 — Which tool saw which stage of the incident

10.1 Correlation method used

- Normalise timestamps and timezones for both tools.
- Filter both tools to the same narrow time window around the test.
- Match the endpoint identity and the network tuple: source 192.168.56.1, destination 192.168.56.40.
- Check that the events are causally related (same actor and sequence) and not merely close in time.
- Document gaps and confidence; do not infer a link from timing alone.

10.2 Correlation findings

Link	Evidence	Confidence
Snort ICMP alert and Wazuh SSH failures share the same source	Same source address in the Snort alert and in Wazuh data.srcip [verify from raw events]	Medium to high once the addresses are confirmed
SSH failures, 5763 and 5715 are one actor	Same source, same account, minutes apart, in rule order 5710/5760 then 5763 then 5715	High
File change is related to the login	Same agent, shortly after 5715/5402; user of the FIM event [confirm]	Medium (timing and host identity; confirm the user in the FIM event)
Snort and the file change	No network signature for a file change	No link claimed (Snort cannot see it)

10.3 Why combining the tools gives better visibility

- **Different vantage points.** Snort saw the ping on the wire; the host never logged it. Wazuh saw the authentication result, the sudo and the file change; Snort cannot see those.
- **Blind spots covered.** SSH is encrypted, so Snort has no visibility into success or failure of a password. A passive host log cannot show that a scan preceded the login.
- **Confidence.** The same source IP appearing in both tools raises confidence that the events belong to one actor.
- **Prioritisation.** Individually, the alerts were Low or Medium. Correlated by source and time, they describe a successful attack path and justify High severity until the source is confirmed authorised.

- **Gap discovery.** Correlation also shows what is missing: Snort raised nothing for the SSH stage, which is a false negative and an action item (Section 15).

11. IOC Investigation and Evidence Register (Task 6)

Only observed values are listed. Lab indicators are labelled as such: an address on the private lab network is context, not proof of a malicious actor. Values still to be copied from raw events are marked.

Indicator	Value (Example)	Importance
IP Address		Identifies source/target of the attack.
Username	root	Helps identify the compromised account.
File Name/Path	/etc/shadow	Shows malicious file or persistence location.
Domain		Indicates C2 or phishing infrastructure.
Port	22	Identifies service/protocol used for communication.
Protocol	TCP	Helps in filtering and tracking traffic.
Timestamp		Shows when the activity occurred.
Wazuh Rule ID	5710	Indicates the type of event and MITRE mapping.
Snort SID		Identifies the specific Snort rule triggered.

Figure 9 — IOC identification table prepared in Week 11 for the failed-login incident (author's work)

IOC type	Value	Source	Importance / caveat
IP address	Source 192.168.56.1 [confirm]; target 192.168.56.40	Snort / Wazuh	Used to search other logs and block the source. Private lab IP, so not a malicious external indicator.
Username	root; standard account ubuntu-wui (lab account)	Wazuh	Account targeted and later used. Lab-only identity; no personal data.
Port	22 (SSH)	Wazuh auth.log / network	Service attacked. Snort did not alert on it.
Protocol	ICMP (recon stage); TCP (SSH stage)	Snort / Wazuh	Helps filter traffic and write the missing SSH rule.
File / path	/tmp/soc-lab-demo/evidence.txt [confirm monitored path]	Wazuh FIM	A path is an indicator, not proof of maliciousness. Harmless test file.
Wazuh rule IDs	5710, 5760, 5711, 5763, 5715, 5501, 5402, 554, 550	Wazuh	Event types and MITRE mapping (T1110 Brute Force for 5763).
Snort SID	1:1000001 “CUSTOM ICMP Ping Detected” (local rule); no SID for SSH	Snort	Shows what the sensor saw and documents the detection gap.
Timestamp	First failure, 5763 alert, 5715 success, FIM events [hh:mm:ss TZ each]	Both	Builds the timeline and shows access duration.

Domain / hash	N/A — none observed (internal traffic only; harmless text file, hash [sha256sum] optional)	—	No command-and-control or exfiltration indicator seen.

11.1 Most important IOC

The most valuable indicator is the pairing **source IP 192.168.56.1 + account root + Wazuh 5715 after 5763**: a successful authentication that follows a brute-force burst from the same source. It is the pivot that turned several Low and Medium alerts into one High-severity incident.

11.2 Evidence handling

- Keep original screenshots and raw event exports unchanged; use the naming E01 to E10 (Appendix B).
- Record collection time, source system, collector and any redaction (the source IP is masked in some screenshots).
- Hash exported files if integrity verification is required: `sha256sum evidence/*`.
- Store evidence in a restricted lab folder; do not upload secrets or personal information.

12. Incident Response (Task 7)

The plan follows the six phases required by the guide. Because this is a controlled exercise, containment may simply be stopping the test and confirming the lab is still isolated; the production-style actions are described so the plan is complete. No destructive remediation is performed on a real or shared system.



Figure 10 — Incident response lifecycle used in the SOC process (Week 11 reference)

12.1 Preparation — what should a SOC have ready?

- Isolated host-only network, approved scope and an up-to-date asset list.
- Synchronised clocks, enabled logs (auth.log, syscheck), snapshots/backups of the VMs.
- Wazuh agents on critical hosts, Snort on the network path, defined L1/L2/L3 roles and escalation contacts.
- Written playbooks (failed logins, port scan, file change) and a ticketing template.

12.2 Identification — how was it detected?

Wazuh rule 5763 raised the brute-force alert, the analyst searched all events from the source IP, found the success (5715), the session (5501), sudo to root (5402) and the FIM events, and confirmed from Snort that only the ICMP stage was visible on the network. Validity was confirmed from raw auth.log lines and the raw alert JSON.

12.3 Containment — what immediate action should be taken?

- Preserve evidence first: export the alerts, copy auth.log, record FIM details and hash them.
- **Lab:** stop the test actions, confirm no further failed logins or file changes, confirm the lab network is still isolated. Record action and time.

- **Production:** block the source IP at the host or network firewall, lock or disable the targeted account, end active sessions, isolate the endpoint if the activity is unauthorised. Wazuh active response was left disabled in the lab, so blocking is a manual step.

12.4 Eradication — how should the root cause be removed?

- Lab: remove only the benign test artefact (/tmp/soc-lab-demo) after the evidence is preserved; keep the log lines.
- Production: reset passwords, review /etc/passwd, /etc/sudoers.d and ~/.ssh/authorized_keys for unknown entries, remove any persistence (accounts, keys, cron jobs).
- Fix the verified weakness: SSH reachable with password authentication and no lockout.

12.5 Recovery — how should the system be restored?

- Restore the lab to its baseline state or snapshot; verify the Wazuh agent shows Active and Snort is still running.
- Confirm expected logs continue (a new post-test event appears in Wazuh).
- Harden SSH: key-only login, PermitRootLogin no, restricted users, fail2ban or equivalent, MFA where possible.
- Monitor the endpoint in the dashboard for several days after the test.

12.6 Lessons learned — how can similar incidents be prevented?

- Host logs caught what the network sensor missed; keep both tools and correlate them.
- Add a Snort rule with a threshold for repeated connections to port 22 and test it with the same controlled sequence.
- Test Wazuh active response in the lab for repeated-failure sources; tune the ping rule to reduce noise.
- Record the exact timestamps, Snort host and FIM path at the time of the test, not afterwards.
- Keep the asset list and playbooks current and review them after every incident.

13. SOC Incident Ticket (Task 8)

Incident ID	DG-SOC-001
Title	Multiple Failed Logins Followed by Successful Login
Date/Time	
Severity	High
Affected System	Ubuntu
Source IP	
User/Account	root
Detection Source	Wazuh
Description	Multiple failed login attempts detected, followed by a successful login from the same IP.
Evidence	Wazuh alert (Rule ID: 5710), system logs, Snort (no related alert).
IOC	
Investigation	Verified logs, confirmed brute force attempt, no malware found.
Action Taken	Blocked IP, reset credentials, increased monitoring.
Current Status	Investigating
Recommendation	Enable MFA, strengthen password policy, and monitor for recurrence.

Figure 11 — Reference: Week 11 ticket DG-SOC-001 in the previous format. The Week 12 ticket below (DG-SOC-2026-001) replaces it; a new screenshot of the final ticket is to be captured (E09).

Final ticket for the capstone incident. Items in brackets must be completed from the raw events before submission.

Field	Value
Source IP	192.168.56.1 (lab source host) [confirm in data.srcip]

Detection Source	Both: Snort (network) and Wazuh (host and FIM)
Wazuh Rule ID	5710, 5760, 5711, 5763 (level 10), 5715, 5501, 5402, 554, 550
Snort SID	1:1000001 “CUSTOM ICMP Ping Detected” (ICMP stage only; no SID for SSH)
Description	Within the isolated lab, a source host probed the endpoint with ICMP, made repeated failed SSH logins crossing the brute-force threshold, then logged in successfully. A session opened, sudo to root ran and a test file in a monitored folder was created and modified.
Indicators of Compromise	Lab indicators (not malicious): source 192.168.56.1, target 192.168.56.40, account root / ubuntu-wui, port 22/TCP, ICMP, path /tmp/soc-lab-demo/evidence.txt, rules and SID above, timestamps [ ]. No domain, hash or malware indicator observed.
Investigation Findings	Sequence confirmed from raw logs and both tools. Source, timing and account match the planned test; no activity outside the plan; no malware; no unexpected file change outside the test folder. Classification: benign true positive.
Containment	Test actions stopped and lab isolation confirmed [time]. In production: block source IP, lock the account, end sessions, isolate the host.
Recovery	Test artefacts removed after evidence was preserved; agent Active and Snort running; new post-test events seen in Wazuh [confirm and attach health checks].
Status	Closed after monitoring period (benign true positive). Escalate instead if the source is ever not on the authorised list.
Recommendations	1) Add a thresholded Snort rule for repeated port-22 connections. 2) Key-only SSH, PermitRootLogin no, MFA. 3) Test Wazuh active response. 4) Tune the ICMP rule. 5) Record timestamps, Snort host and FIM path at test time.

14. SOC Metrics (Task 9)

Two datasets are used and kept separate so the counts stay consistent. **Dataset A** is the Wazuh dashboard overview for the last 24 hours (Figure 2). **Dataset B** is the set of alert types reviewed for this incident (Section 8.2). Counts of individual alert instances should be completed from a Discover query before submission.

Metric	Value	Definition / calculation
Total Wazuh alerts, 24 h window (Dataset A)	123	Dashboard: 0 + 0 + 50 + 73 by rule-level band
Total events analysed (Dataset B)	[n] events	Unique source events in the test window; count with a Discover filter on agent and time
Alert types reviewed (Dataset B)	10	1 Snort signature + 9 Wazuh rules
True positives	10 alert types (benign TP, authorised)	Each rule detected the behaviour it was designed to detect
False positives	0 alert types	No alert was ruled out after evidence-based review
Inconclusive	0	All required fields available except masked values to be completed
Critical (analyst)	0	Rubric below
High (analyst)	2	5763, 5715
Medium (analyst)	6	5710, 5760, 5711, 5402, 554, 550

Low (analyst)	2	Snort 1:1000001, 5501
Wazuh dashboard bands (Dataset A)	Critical 0, High 0, Medium 50, Low 73	Rule-level bands from Figure 2

Metric	Value	Definition / calculation
Snort detections	[n] alerts (1 signature)	`grep -c "1000001"` on the Snort alert log for the window
Wazuh detections	9 rules; [n] alerts	Discover filter agent.name: ubuntu-agent for the window
Most important IOC	192.168.56.1 + root + 5715 after 5763	Successful login after brute-force burst from one source

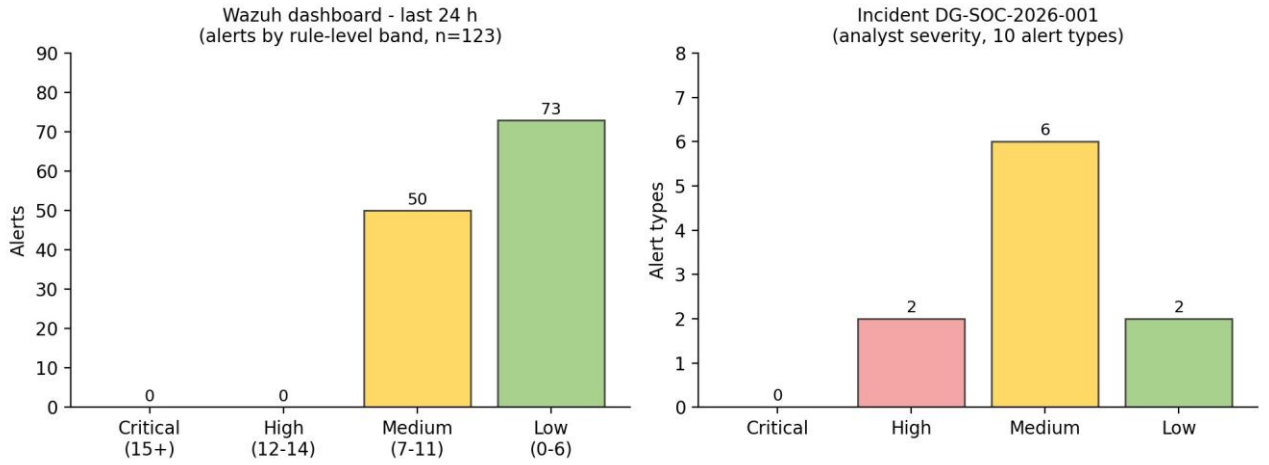


Figure 12 — Left: Wazuh dashboard alerts by rule-level band (Dataset A). Right: analyst severity for the incident's 10 alert types (Dataset B).

14.1 Severity rubric used

Severity	Decision principle
Critical	Credible evidence of severe compromise or immediate high impact.
High	Strong evidence of significant unauthorised activity or a high-risk condition.
Medium	Suspicious activity requiring investigation, impact not yet confirmed.
Low	Limited-risk event or low-confidence signal requiring routine review.
Informational	Expected or benign activity useful for context.

15. Lessons Learned and Analyst Reflection (Task 10)

What I learned in the four-week SOC module

Snort and IDS. In Week 9 I learned that an IDS only reports what a rule describes. My custom ICMP rule (SID 1:1000001) fired reliably, which showed me how generator, SID and revision identify a signature, but it also showed the limit of network visibility: with no rule for SSH, password guessing was invisible to Snort, and encrypted payloads cannot be read in any case. A running sensor is not proof of coverage.

Wazuh and SIEM. In Week 10 I built the manager, indexer, dashboard and an agent. The most useful lesson was that the SIEM adds history and context: frequency rule 5763 turned many 5760 failures into one brute-force alert, and the Security events and Discover views let me pivot on source IP, account and time.
File Integrity Monitoring added endpoint evidence that no network tool can provide.

Alert and log analysis. I learned to open the raw event before trusting the summary: agent, rule ID and level, user, source IP and log location. I also learned that tool severity and analyst severity differ. Rule 5715 is a level-3 event, but in sequence after 5763 it becomes the most important alert in the incident.

IOCs. Useful indicators were the source IP, the account, port 22, the rule IDs and SID, the file path and the timestamps. Context was essential: a private lab IP or a path is an indicator, not proof of maliciousness, and unknown values must be marked N/A rather than guessed.

Incident response and L1 responsibilities. I practised the lifecycle from preparation to lessons learned and wrote a ticket and three playbooks. As an L1 analyst I would acknowledge the alert, validate the raw fields, check related events, classify (including the benign true positive category), rate severity using impact, confidence and asset criticality, escalate on success after failures or privileged activity, and record every action while preserving evidence.

Challenges I faced. The main detection issue was that Snort produced no alert for SSH guessing; I diagnosed it by checking which rules were loaded and realising my Week 9 rule set covers only ICMP and HTTP. A second challenge was evidence quality: the source IP is masked in several screenshots and the dashboard overview showed no registered agent while alerts were arriving, so I learned to confirm values in the raw JSON and to capture the agent page as its own evidence.

How I would improve the SOC environment

- Add a thresholded Snort rule for repeated connections to port 22 and test it.
- Move Snort to its own VM, separate from the traffic source, so the sensor is independent.
- Enforce NTP across all VMs and record timezones so correlation is reliable.
- Test Wazuh active response for automated blocking of repeated-failure sources.
- Widen FIM scope to /etc and SSH keys and tune noisy rules with thresholds.
- Keep the asset list and playbooks current and review them after every incident.